

Week 2

Name: Dhaksin Kaarthick S.U

Rollno: CH.SC.U4CSE24115

Sort

1. Bubble Sort

Code:

```
# include <stdio.h>
# define n 10
int main() {
int arr[n]={5,3,8,1,2,7,9,4,6,0};

for(int i=0;i<n-1;i++){
for(int j=i+1;j<n;j++){
if (arr[i]>arr[j]){
int temp=arr[i];
arr[i]=arr[j];
arr[j]=temp;
}
}

}

for (int i=0;i<n;i++){
printf("%d ",arr[i]);
}
}
```

Output:

```
D:\Dhaksin\DAA\week 2>gcc bubble.c
D:\Dhaksin\DAA\week 2>a
0 1 2 3 4 5 6 7 8 9
```

Time Complexity

- **Worst Case:** $O(n^2)$ – every pair of elements is compared and swapped.

Justification:

Bubble sort uses two nested loops; each pass compares adjacent elements, leading to quadratic comparisons.

Space Complexity

- **$O(1)$** (In-place)

Justification:

Only a temporary variable is used for swapping.

2. Insertion Sort:

Code:

```
# include <stdio.h>
# define n 10
int main() {
int arr[n]={5,3,8,1,2,7,9,4,6,0};
for (int i=1;i<n;i++){
int j=i-1;
int key=arr[i];
while (j>=0 && arr[j]>key){
arr[j+1]=arr[j];
j--;
}
arr[j+1]=key;
}
for (int i=0;i<n;i++){
printf("%d ",arr[i]);
}
}
```

Output:

```
D:\Dhaksin\DAA\week 2>gcc insertion.c
D:\Dhaksin\DAA\week 2>a
0 1 2 3 4 5 6 7 8 9
```

Justification:

Space Complexity

- $O(1)$ (In-place)
- Sorting is done within the original array using a single key variable

Time Complexity

- Worst Case: $O(n^2)$ – elements are in reverse order.
- Each element may need to be compared and shifted across the sorted portion of the array.

3. Selection Sort:

Code:

```
# include <stdio.h>
# define n 10
int main() {
int arr[n]={5,3,8,1,2,7,9,4,6,0};
for (int i=0;i<n-1;i++){
int min=i;
for (int j=i+1;j<n;j++){
if (arr[min]>arr[j]){
min=j;
}
}
if (min!=i){
int temp=arr[i];
arr[i]=arr[min];
arr[min]=temp;
}
}
for (int i=0;i<n;i++){
printf("%d ",arr[i]);
}
}
```

Output:

```
D:\Dhaksin\DAA\week 2>gcc selection.c
D:\Dhaksin\DAA\week 2>a
0 1 2 3 4 5 6 7 8 9
D:\Dhaksin\DAA\week 2>|
```

Time Complexity

- Best / Average / Worst Case: $O(n^2)$

Justification:

For every element, the algorithm scans the remaining unsorted part to find the minimum, resulting in fixed quadratic comparisons.

Space Complexity

- $O(1)$ (In-place)

Justification:

Only one temporary variable is required for swapping.

4.Heap Sort (max)

Code:

```
#include <stdio.h>

void maxHeapify(int arr[], int heapSize, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heapSize && arr[left] > arr[largest])
        largest = left;
    if (right < heapSize && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int tmp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = tmp;
        maxHeapify(arr, heapSize, largest);
    }
}

void buildMaxHeap(int arr[], int n) {
    for (int i = (n / 2) - 1; i >= 0; i--)
        maxHeapify(arr, n, i);
}

void heapSortMax(int arr[], int n) {
    buildMaxHeap(arr, n);
    for (int i = n - 1; i >= 1; i--) {
        // Move current max to end
        int tmp = arr[0];
        arr[0] = arr[i];
        arr[i] = tmp;
        // Heapify reduced heap
        maxHeapify(arr, i, 0);
    }
}
```

```
int main() {
    int arr[] = {42, 23, 4, 16, 8, 15};
    int n = sizeof(arr)/sizeof(arr[0]);

    heapSortMax(arr, n);

    printf("Sorted (ascending) using max-heap heapsort:\n");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
```

Output:

```
D:\Dhaksin\DAA\week 2>gcc heapmax.c
D:\Dhaksin\DAA\week 2>a
Sorted (ascending) using max-heap heapsort:
4 8 15 16 23 42
```

5.Heap sort (Min)

Code:

```
#include <stdio.h>

void minHeapify(int arr[], int heapSize, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < heapSize && arr[left] < arr[smallest])
        smallest = left;
    if (right < heapSize && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        int tmp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = tmp;
        minHeapify(arr, heapSize, smallest);
    }
}

void buildMinHeap(int arr[], int n) {
    for (int i = (n / 2) - 1; i >= 0; i--)
        minHeapify(arr, n, i);
}

void heapSortMin(int arr[], int n) {
    buildMinHeap(arr, n);
    for (int i = n - 1; i >= 1; i--) {
        |
        int tmp = arr[0];
        arr[0] = arr[i];
        arr[i] = tmp;

        minHeapify(arr, i, 0);
    }
}
```

```

int main() {
    int arr[] = {42, 23, 4, 16, 8, 15};
    int n = sizeof(arr)/sizeof(arr[0]);

    heapSortMin(arr, n);

    printf("Sorted (descending) using min-heap heapsort:\n");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

Output:

```

D:\Dhaksin\DAA\week 2>gcc heapmin.c
D:\Dhaksin\DAA\week 2>a
Sorted (descending) using min-heap heapsort:
42 23 16 15 8 4

```

Time Complexity

- Insertion: $O(\log n)$
- Deletion: $O(\log n)$
- Finding Min (Min Heap) / Max (Max Heap): $O(1)$

Justification:

Heap height is $\log n$, and operations involve moving elements up or down the heap.

Space Complexity

- $O(1)$ (array-based implementation)

Justification:

Heap uses the same array for storage

6. Bucket Sort:

Code:

```
#include <stdio.h>
#define BUCKETS 10
#define MAX_BUCKET_SIZE 20

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void bucketSort(int arr[], int n) {
    int buckets[BUCKETS][MAX_BUCKET_SIZE];
    int bucketCount[BUCKETS] = {0};
    int max = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }

    for (int i = 0; i < n; i++) {
        int index = (arr[i] * BUCKETS) / (max + 1);
        buckets[index][bucketCount[index]++] = arr[i];
    }

    int k = 0;
    for (int i = 0; i < BUCKETS; i++) {
        if (bucketCount[i] > 0) {
            insertionSort(buckets[i], bucketCount[i]);

            for (int j = 0; j < bucketCount[i]; j++) {
                arr[k++] = buckets[i][j];
            }
        }
    }
}
```

```
int main() {
    int arr[] = {42, 23, 4, 16, 8, 15};
    int n = sizeof(arr) / sizeof(arr[0]);

    bucketSort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```


Output:

```
D:\Dhaksin\DAA\week 2>gcc bucketsort.c
D:\Dhaksin\DAA\week 2>a
Sorted array: 4 8 15 16 23 42
```

Time Complexity

- Worst Case: $O(n^2)$ (when all elements fall into one bucket)

Justification:

Elements are distributed into buckets and sorted individually. Performance depends on uniform distribution.

Space Complexity

- $O(n + k)$

Justification:

Extra memory is required to store buckets and their elements.

Output:

Graph:

1.Depth First Search:

Code:

```
#include <stdio.h>

int visited[10];
int adj[10][10];
int n;

void dfs(int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (adj[v][i] == 1 && visited[i] == 0) {
            dfs(i);
        }
    }
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    int start;
    printf("Enter starting vertex: ");
    scanf("%d", &start);

    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("DFS traversal: ");
    dfs(start);

    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 0
0 0 1 0 0
Enter starting vertex: 0
DFS traversal: 0 1 3 2 4
```

Time Complexity

- $O(V + E)$

Justification:

Each vertex and each edge is visited exactly once.

Space Complexity

- $O(V)$

Justification:

Recursive stack (or explicit stack) stores vertices during traversal.

2. Breadth First Search:

Code:

```
#include <stdio.h>
int visitedB[10];
int adjB[10][10];
int nB;
void bfs(int start) {
    int queue[20], front = 0, rear = 0;
    queue[rear++] = start;
    visitedB[start] = 1;
    while (front < rear) {
        int v = queue[front++];
        printf("%d ", v);
        for (int i = 0; i < nB; i++) {
            if (adjB[v][i] == 1 && visitedB[i] == 0) {
                queue[rear++] = i;
                visitedB[i] = 1;
            }
        }
    }
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &nB);

    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < nB; i++)
        for (int j = 0; j < nB; j++)
            scanf("%d", &adjB[i][j]);

    int start;
    printf("Enter starting vertex: ");
    scanf("%d", &start);

    for (int i = 0; i < nB; i++)
        visitedB[i] = 0;

    printf("BFS traversal: ");
    bfs(start);

    return 0;
}
```

Output:

```
D:\Dhaksin\DAA\week 2>gcc bfs.c
D:\Dhaksin\DAA\week 2>a
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 0
0 0 1 0 0
Enter starting vertex: 0
BFS traversal: 0 1 2 3 4
```

Time Complexity

- $O(V + E)$

Justification:

All vertices and edges are explored once using a queue.

Space Complexity

- $O(V)$

Justification:

Queue stores all vertices at the current level of traversal.